

零声教育 Darren

课程：[埃梯零声 \(0voice.com\)](https://www.0voice.com/)

特别注意：

课程使用的RO2版本必须在 **Ubuntu24.04 desktop 桌面版**运行

Ubuntu24.04桌面版下载地址：<https://mirrors.aliyun.com/ubuntu-releases/24.04/ubuntu-24.04.2-desktop-amd64.iso>

课件地址：https://gitlab.0voice.com/2503_vip/ros2-doc

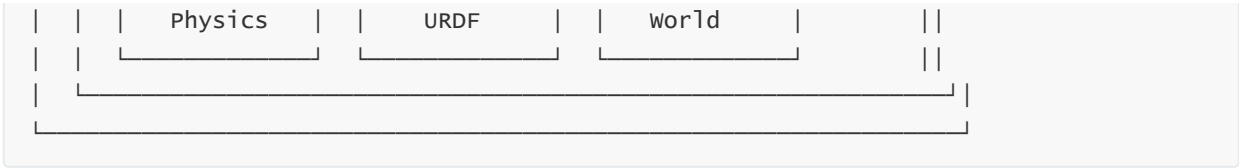
源码地址：https://gitlab.0voice.com/2503_vip/ros2-src

从本节课开始，我们从0实现一个两轮差速机器人项目。

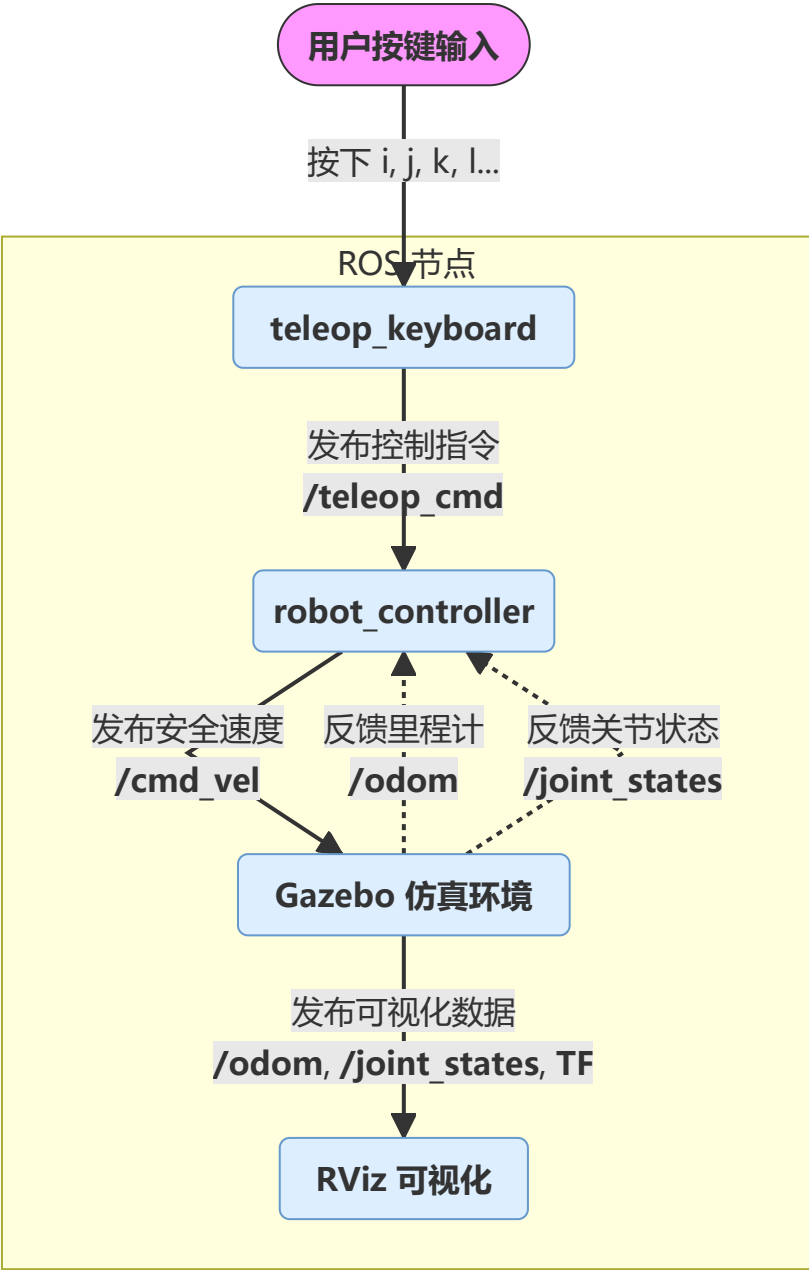
1 项目架构设计

1.1 系统架构图





1.2 数据流向图



1.3 项目创建与配置

安装ROS-Gazebo桥接

```
sudo apt install ros-humble-ros-gz-sim ros-humble-ros-gz-bridge -y
```

创建ROS2包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake differential_drive_robot \
  --dependencies rclcpp geometry_msgs sensor_msgs nav_msgs tf2 tf2_geometry_msgs
```

项目目录结构

```
cd differential_drive_robot
mkdir -p launch rviz urdf worlds srv doc
```

最终目录结构：

```
differential_drive_robot/
├─ CMakeLists.txt          # CMake构建配置
├─ package.xml             # 包依赖声明
├─ README.md               # 项目说明
├─ doc/                    # 文档目录
├─ include/                # 头文件目录
├─ launch/                 # 启动文件
├─ rviz/                   # RViz配置
├─ src/                    # 源代码
├─ srv/                    # 服务定义
├─ urdf/                   # 机器人模型
└─ worlds/                 # 仿真世界
```

2 核心组件实现

2.1 机器人建模 (URDF)

URDF (Unified Robot Description Format) 是ROS中描述机器人模型的标准格式。它是一个XML文件，定义了机器人的物理结构和特性。

文件位置：urdf/robot_harmonic.urdf

2.1.1 URDF核心概念解析

框架：

```
<?xml version="1.0"?>
<robot name="differential_drive_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">
.....填充
</robot>
```

文件的开头是: <robot name="differential_drive_robot" ...>

文件的结尾是:

重要元素:

- **<robot>**: URDF文件的根元素, 所有内容都在这个标签内。
- **<link>**: 定义机器人的一个刚体部分, 如底盘、轮子。它有三个核心子标签:
 - **<visual>**: 定义link的外观 (形状、颜色、材质) 。
 - **<collision>**: 定义link的碰撞几何体, 用于物理仿真。
 - **<inertial>**: 定义link的动力学属性 (质量、惯性张量) 。
- **<joint>**: 定义两个link之间的连接关系, 描述它们的相对运动。关键属性:
 - **type**: 关节类型, 如 **fixed** (固定), **continuous** (无限旋转), **revolute** (有限旋转), **prismatic** (滑动)。
 - **<parent>** 和 **<child>**: 定义连接的两个link。
 - **<origin>**: 定义子link相对于父link的坐标变换 (位置和姿态) 。
 - **<axis>**: 定义关节的运动轴。

总的来说, 您可以把URDF文件看作是一个以 为根的XML文档, 它通过一系列嵌套的 和 标签, 像搭积木一样, 精确地描述出一个机器人的完整物理结构。

2.1.2 材质定义

```
<!-- ===== 材质定义 ===== -->
<!-- 蓝色材质 - 用于机器人底盘 -->
<material name="blue">
  <color rgba="0.2 0.2 1.0 1.0"/>
</material>

<!-- 黑色材质 - 用于驱动轮胎 -->
<material name="black">
  <color rgba="0.1 0.1 0.1 1.0"/>
</material>

<!-- 白色材质 - 用于万向轮 -->
<material name="white">
  <color rgba="0.9 0.9 0.9 1.0"/>
</material>
```

2.1.3 底盘 (base_link)

底盘是机器人的核心，所有其他组件都连接到它上面。

```
<!-- ===== 机器人底盘 ===== -->
<!--
    底盘是机器人的主体结构，承载所有其他组件
    尺寸：长0.6m × 宽0.4m × 高0.2m
    质量：15kg（包含电池、控制器等设备重量）
    材质：蓝色，便于在仿真中识别
-->
<link name="base_link">
    <!-- 视觉外观定义 -->
    <visual>
        <origin xyz="0 0 0.1" rpy="0 0 0"/> <!-- 底盘中心位置 -->
        <geometry>
            <box size="0.6 0.4 0.2"/> <!-- 长方体底盘 -->
        </geometry>
        <material name="blue"/> <!-- 蓝色外观 -->
    </visual>

    <!-- 碰撞检测几何 -->
    <collision>
        <origin xyz="0 0 0.1" rpy="0 0 0"/>
        <geometry>
            <box size="0.6 0.4 0.2"/> <!-- 与视觉模型相同 -->
        </geometry>
    </collision>

    <!-- 惯性属性 - 影响物理仿真的动力学行为 -->
    <inertial>
        <origin xyz="0 0 0.1" rpy="0 0 0"/>
        <mass value="15.0"/> <!-- 底盘质量 -->
        <inertia ixx="0.1" ixy="0.0" ixz="0.0"
            iyy="0.1" iyz="0.0" izz="0.1"/> <!-- 惯性张量 -->
    </inertial>
</link>
```

设计说明:

- `origin` 的Z值为0.1，因为box的高度是0.2，这样几何中心就在原点。
- `inertial` 是物理仿真的关键，不正确的惯性参数会导致机器人行为异常。

2.1.4 驱动轮 (left_wheel & right_wheel)

以左轮为例，右轮对称。

```
<!-- ===== 左驱动轮 ===== -->
<!--
    左侧驱动轮，通过continuous关节连接到底盘
-->
```

半径: 0.1m, 宽度: 0.05m
质量: 2kg (包含电机重量)
位置: 底盘左侧, 距离中心0.225m

```
-->
<link name="left_wheel">
  <!-- 视觉外观 - 黑色圆柱体 -->
  <visual>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/> <!-- 绕X轴旋转90度, 轮子朝向正确 -->
    <geometry>
      <cylinder radius="0.1" length="0.05"/> <!-- 圆柱形轮子 -->
    </geometry>
    <material name="black"/>
  </visual>

  <!-- 碰撞检测 -->
  <collision>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry>
      <cylinder radius="0.1" length="0.05"/>
    </geometry>
  </collision>

  <!-- 惯性属性 -->
  <inertial>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <mass value="2.0"/>
    <inertia ixx="0.005" ixy="0.0" ixz="0.0"
      iyy="0.005" iyz="0.0" izz="0.01"/>
  </inertial>
</link>

<!-- 左轮关节 - 连接底盘和左轮 -->
<joint name="left_wheel_joint" type="continuous">
  <parent link="base_link"/> <!-- 父链接: 底盘 -->
  <child link="left_wheel"/> <!-- 子链接: 左轮 -->
  <origin xyz="0.0 0.225 0.0" rpy="0 0 0"/> <!-- 关节位置: 底盘左侧 -->
  <axis xyz="0 1 0"/> <!-- 旋转轴: Y轴方向 (轮子转动方向) -->
  <dynamics damping="0.1" friction="0.1"/> <!-- 阻尼和摩擦参数 -->
</joint>

<!-- ===== 右驱动轮 ===== -->
<!-- 右侧驱动轮, 与左轮对称 -->
<link name="right_wheel">
  <visual>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry>
      <cylinder radius="0.1" length="0.05"/>
    </geometry>
    <material name="black"/>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
```

```

    <geometry>
      <cylinder radius="0.1" length="0.05"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <mass value="2.0"/>
    <inertia ixx="0.005" ixy="0.0" ixz="0.0"
              iyy="0.005" iyz="0.0" izz="0.01"/>
  </inertial>
</link>

<!-- 右轮关节 -->
<joint name="right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="right_wheel"/>
  <origin xyz="0.0 -0.225 0.0" rpy="0 0 0"/> <!-- 底盘右侧 -->
  <axis xyz="0 1 0"/>
  <dynamics damping="0.1" friction="0.1"/>
</joint>

```

设计说明:

- `type="continuous"`: 驱动轮需要无限旋转。
- `origin: xyz="0.0 0.225 0.0"` 将轮子放置在Y轴正方向0.225m处。轮间距为0.45m, 所以轮子中心距离机器人中心线为0.225m。
- `axis: xyz="0 1 0"` 定义了轮子的旋转轴是Y轴。

2.1.5 前后万向轮 (`front_caster` & `rear_caster`)

万向轮用于支撑, 我们将其简化为球体, 并使用 `fixed` 关节固定在底盘上。

```

<!-- ===== 前万向轮 ===== -->
<!--
  前万向轮, 提供前端支撑, 防止机器人前倾
  球形设计, 摩擦力设置为0, 不影响差速驱动
  位置: 底盘前端, 距离中心0.2m
-->
<link name="front_caster">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <sphere radius="0.05"/> <!-- 球形万向轮 -->
    </geometry>
    <material name="white"/>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <sphere radius="0.05"/>

```

```

    </geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="0.5"/> <!-- 较轻的万向轮 -->
    <inertia ixx="0.001" ixy="0.0" ixz="0.0"
              iyy="0.001" iyz="0.0" izz="0.001"/>
  </inertial>
</link>

<!-- 前万向轮关节 - 固定连接 -->
<joint name="front_caster_joint" type="fixed">
  <parent link="base_link"/>
  <child link="front_caster"/>
  <origin xyz="0.2 0.0 -0.05" rpy="0 0 0"/> <!-- 底盘前端，略低于底盘底面 -->
</joint>

<!-- ===== 后万向轮 ===== -->
<!-- 后万向轮，与前万向轮对称 -->
<link name="rear_caster">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <sphere radius="0.05"/>
    </geometry>
    <material name="white"/>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <sphere radius="0.05"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="0.5"/>
    <inertia ixx="0.001" ixy="0.0" ixz="0.0"
              iyy="0.001" iyz="0.0" izz="0.001"/>
  </inertial>
</link>

<!-- 后万向轮关节 -->
<joint name="rear_caster_joint" type="fixed">
  <parent link="base_link"/>
  <child link="rear_caster"/>
  <origin xyz="-0.2 0.0 -0.05" rpy="0 0 0"/> <!-- 底盘后端 -->
</joint>

```

设计说明:

- `type="fixed"`: 万向轮与底盘没有相对运动。
- **摩擦力**: 在Gazebo配置中，我们会将万向轮的摩擦力设置为0。

2.1.6 集成Gazebo插件 - 赋予机器人“生命”

URDF本身只描述几何和物理属性，为了让机器人在Gazebo中动起来，需要使用 `<gazebo>` 标签添加插件。

差速驱动插件 (`gz-sim-diff-drive-system`)

这是最重要的插件，它使机器人能够响应速度命令。

```
<!-- ===== Gazebo插件配置 ===== -->

<!--
    差速驱动插件 - 核心运动控制插件
    功能：
    1. 接收cmd_vel话题的速度命令
    2. 控制左右轮的转速实现差速驱动
    3. 发布odom话题提供里程计信息
    4. 发布TF变换 (odom -> base_link)

    这是robot_controller.cpp必需的插件
-->
<gazebo>
  <plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">
    <!-- 轮子关节配置 -->
    <left_joint>left_wheel_joint</left_joint>    <!-- 左轮关节名称 -->
    <right_joint>right_wheel_joint</right_joint>  <!-- 右轮关节名称 -->

    <!-- 机器人几何参数 -->
    <wheel_separation>0.45</wheel_separation>    <!-- 轮间距 (m) -->
    <wheel_radius>0.1</wheel_radius>              <!-- 轮半径 (m) -->

    <!-- 里程计发布频率 -->
    <odom_publish_frequency>30</odom_publish_frequency> <!-- 30Hz发布频率 -->

    <!-- 速度限制 - 与robot_controller.cpp中的限制保持一致 -->
    <max_linear_velocity>1.0</max_linear_velocity>    <!-- 最大线速度 1.0 m/s -->
    <max_angular_velocity>2.0</max_angular_velocity>  <!-- 最大角速度 2.0 rad/s -->
  ->
    <min_linear_velocity>-1.0</min_linear_velocity>  <!-- 最小线速度 -1.0 m/s -->
  >
    <min_angular_velocity>-2.0</min_angular_velocity> <!-- 最小角速度 -2.0 rad/s -->
  -->

  <!-- ROS2话题配置 -->
  <topic>cmd_vel</topic>    <!-- 订阅的速度命令话题 -->
  <odom_topic>odom</odom_topic> <!-- 发布的里程计话题 -->

  <!-- 坐标系配置 -->
  <frame_id>odom</frame_id>    <!-- 里程计坐标系ID -->
  <child_frame_id>base_link</child_frame_id> <!-- 机器人坐标系ID -->
  <tf_topic>/tf</tf_topic>    <!-- TF变换话题 -->
</plugin>
</gazebo>
```

设计说明:

- 此插件订阅 `/cmd_vel` 话题，根据线速度和角速度计算左右轮的目标转速。
- 同时，它还根据轮子转速计算里程计信息，并发布到 `/odom` 话题。

关节状态发布插件 (`gz-sim-joint-state-publisher-system`)

此插件读取所有关节的当前状态（位置、速度）并发布到 `/joint_states` 话题。

```
<!--
  关节状态发布插件
  功能：发布 joint_states 话题，包含所有关节的位置、速度、力矩信息
  robot_controller.cpp 使用此信息获取左右轮的角速度
-->
<gazebo>
  <plugin filename="gz-sim-joint-state-publisher-system"
name="gz::sim::systems::JointStatePublisher">
    <topic>joint_states</topic> <!-- 发布的关节状态话题 -->
  </plugin>
</gazebo>
```

设计说明:

- `robot_state_publisher` 节点会订阅此话题，结合 URDF 中的模型信息，计算并发布所有 link 之间的 TF 变换。
- `robot_controller` 也订阅此话题，用于监控轮子的实际速度。

2.2 创建仿真世界 (SDF) - 搭建“舞台”

SDF (Simulation Description Format) 是 Gazebo 原生使用的世界描述格式。它比 URDF 功能更强大，可以定义光照、物理引擎、地形等非机器人元素。

相关文件 `worlds/empty_harmonic.sdf`

框架:

```
<sdf version="1.6">
  <world name="empty">
    ....填充
  </world>
</sdf>
```

2.2.1 物理引擎配置

```
<physics name="1ms" type="ignored">
  <max_step_size>0.001</max_step_size>
  <real_time_factor>1.0</real_time_factor>
</physics>
```

设计说明:

- `max_step_size`: 0.001 秒 (1ms) 的步长提供了较高的仿真精度。
- `real_time_factor`: 1.0 表示仿真时间与真实时间同步。

2.2.2 光照系统

```
<light name="sun" type="directional">
  <cast_shadows>true</cast_shadows>
  <pose>0 0 10 0 0 0</pose>
  <diffuse>0.8 0.8 0.8 1</diffuse>
  <specular>0.2 0.2 0.2 1</specular>
</light>
```

设计说明: `directional` 类型的光源模拟了太阳光，提供了均匀的平行光照。

2.2.3 地面模型

```
<model name="ground_plane">
  <static>true</static>
  <link name="link">
    <collision name="collision">
      <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
    </collision>
    <visual name="visual">
      <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
      <material><ambient>0.8 0.8 0.8 1</ambient>...</material>
    </visual>
  </link>
</model>
```

设计说明:

- `<static>true</static>`: 将地面设置为静态模型，物理引擎不会计算它的运动，极大地提高了性能。
- `<plane>`: 使用无限大的平面作为地面。

2.3 配置可视化界面 (RViz) - 连接“现实”与“虚拟”

rviz/robot_view.rviz

RViz是ROS的3D可视化工具，它通过订阅各种话题（如TF, Odometry等）来显示机器人状态。

2.3.1 全局选项 (Global Options)

```
Global Options:  
Background Color: 48; 48; 48  
Fixed Frame: odom  
Frame Rate: 30
```

设计说明:

- **Fixed Frame**: 这是RViz中 **最重要的设置**。它定义了所有数据的参考坐标系。我们选择 `odom`，因为它是一个固定的、连续增长的世界坐标系。如果选择 `base_link`（机器人自身的坐标系），那么整个世界看起来都会随着机器人的移动而移动。

2.3.2 显示插件 (Displays)

- **Grid (网格)**: 提供一个参考平面，帮助我们感知机器人的位置和尺度。
- **RobotModel (机器人模型)**:
 - **Description Source: Topic**: 告诉RViz从 `/robot_description` 话题获取URDF模型。
 - **Description Topic**: `/robot_description: robot_state_publisher` 节点发布的话题。
- **TF (坐标变换)**:
 - **作用**: 可视化所有link之间的坐标系关系，形成TF树。
 - **调试价值**: 如果机器人的某个部件位置不正确，首先要检查TF树是否如预期。
- **Odometry (里程计)**:
 - **Topic**: `/odom`: 订阅差速驱动插件发布的 `/odom` 话题。
 - **Keep**: 50: 在屏幕上保留最近50个位姿，形成一条运动轨迹。
 - **Shape: Arrow**: 在机器人的当前位置显示一个箭头，表示其姿态。

2.4 编写控制逻辑 (C++) 与 整合系统 (Launch)

这部分将详细解释C++代码和Launch文件。

2.4.1 控制器 `robot_controller.cpp`

该节点是机器人的"大脑"，负责接收遥控命令并将其转化为给Gazebo的实际速度指令。

关键功能:

- **订阅 `teleop_cmd`**: 接收来自键盘遥控节点的 `Twist` 消息。
- **安全检查**:
 - **速度限制**: 使用 `std::clamp` 确保请求的速度不超过预设的最大值，防止机器人失控。
 - **超时机制**: 创建一个定时器，如果超过1秒没有收到新的遥控命令，就自动发布速度为0的指令，让机器人停止。这是非常重要的安全功能。
- **发布 `cmd_vel`**: 将处理过的安全速度指令发布出去，供Gazebo的差速驱动插件订阅。

- **订阅 odom 和 joint_states**: 接收机器人的状态反馈，虽然在当前简化版中主要用于日志记录，但为未来的高级控制（如PID闭环控制）打下了基础。

2.4.2 遥控节点 teleop_keyboard.cpp

该节点负责将用户的键盘按键转换为ROS的 `Twist` 消息。

关键技术:

- **非阻塞终端I/O**: 使用 `termios.h` C库来配置终端，使其进入非缓冲、不回显的模式。这样程序就可以立即读取单个按键，而无需用户按回车。
- **高频检测**: 通过一个50Hz的ROS定时器（`create_wall_timer`）循环检测键盘输入，保证了控制的实时性。

2.4.3 启动文件 gazebo_harmonic.launch.py

文件位置: `launch/gazebo_harmonic.launch.py`

这是将所有分散的节点和配置整合起来的核心。

启动顺序与内容:

1. **设置环境变量**: `SetEnvironmentVariable` 确保Gazebo能找到我们的模型文件。
2. **启动Gazebo**: `IncludeLaunchDescription` 复用官方的 `gz_sim.launch.py` 来启动仿真器，并传入我们的世界文件 `empty_harmonic.sdf`。
3. **启动 robot_state_publisher**: 该节点读取 `/robot_description` 话题的URDF内容，并根据 `/joint_states` 话题计算和发布所有link之间的TF变换。
4. **启动 ros_gz_bridge**: 建立ROS话题与Gazebo话题之间的桥梁，如将ROS的 `/cmd_vel` 转换为Gazebo能理解的速度命令。
5. **生成机器人实体**: `ros_gz_sim create` 节点将URDF模型在Gazebo中实例化。
6. **启动 robot_controller**: 运行我们的主控制节点。
7. **启动 rviz2**: 运行可视化工具，并使用 `-d` 参数加载我们的配置文件。

2.5 配置文件

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(differential_drive_robot)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
```

```
# 依赖包
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(nav_msgs REQUIRED)
find_package(tf2 REQUIRED)
find_package(tf2_ros REQUIRED)
find_package(tf2_geometry_msgs REQUIRED)
# Gazebo Harmonic相关包 - 如果可用则使用
find_package(ros_gz_sim QUIET)
find_package(ros_gz_bridge QUIET)
find_package(rosidl_default_generators REQUIRED)

# 机器人控制器可执行文件
add_executable(robot_controller src/robot_controller.cpp)
ament_target_dependencies(robot_controller
  rclcpp
  geometry_msgs
  sensor_msgs
  nav_msgs
  tf2
  tf2_ros
  tf2_geometry_msgs
)

# 键盘遥控可执行文件
add_executable(teleop_keyboard src/teleop_keyboard.cpp)
ament_target_dependencies(teleop_keyboard
  rclcpp
  geometry_msgs
)

# 安装可执行文件
install(TARGETS
  robot_controller
  teleop_keyboard
  DESTINATION lib/${PROJECT_NAME}
)

# 安装文件夹
install(DIRECTORY
  urdf
  launch
  worlds
  rviz
  DESTINATION share/${PROJECT_NAME}
)
```

```

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source files
  set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # comment the line when this package is in a git repo and when
  # a copyright and license is added to all source files
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()

```

package.xml

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>differential_drive_robot</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="326873713@qq.com">lqf</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>rc1cpp</depend>
  <depend>geometry_msgs</depend>
  <depend>sensor_msgs</depend>
  <depend>nav_msgs</depend>
  <depend>tf2</depend>
  <depend>tf2_geometry_msgs</depend>

  <!-- URDF和可视化 -->
  <depend>urdf</depend>
  <depend>xacro</depend>
  <depend>robot_state_publisher</depend>
  <depend>joint_state_publisher</depend>
  <depend>rviz2</depend>

  <!-- Gazebo Harmonic仿真 -->
  <depend>ros_gz_sim</depend>
  <depend>ros_gz_bridge</depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

```

```
<export>
  <build_type>ament_cmake</build_type>
</export>
</package>
```

3 构建与运行

3.1 编译项目

```
cd ~/ros2_ws
colcon build --packages-select differential_drive_robot
source install/setup.bash
```

3.2 运行系统

```
# 启动完整仿真系统（Gazebo + RViz + 控制器）
ros2 launch differential_drive_robot gazebo_harmonic.launch.py

# 在新终端启动键盘控制
ros2 run differential_drive_robot teleop_keyboard
```

3.3 验证功能

- 检查Gazebo中机器人是否正确显示。
- 验证RViz可视化是否正常。
- 测试键盘控制响应。
- 观察话题通信状态。

3.4 调试技巧

检查节点状态

```
ros2 node list
```

检查话题列表

```
ros2 topic list
```

监听话题数据

```
ros2 topic echo /cmd_vel
```

```
ros2 topic echo /odom
```

检查TF树

```
ros2 run tf2_tools view_frames.py
```

```
evince frames.pdf
```
